

Using unknown references in constant expressions

Document #: P2280R0
Date: 2021-01-13
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Introduction](#)

[1.1 Wait, why?](#)

[1.2 Other Examples](#)

[2 Proposal](#)

[2.1 Implementation Experience](#)

[2.2 Other not-quite-reference examples](#)

[2.3 Lifetime Dilemma](#)

[2.4 Still further cases](#)

[2.5 Wording](#)

[3 Acknowledgments](#)

§ 1 Introduction

Let's say I have an array and want to get its size as a constant expression. In C, I had to write a macro:

```
#define ARRAY_SIZE(a) (sizeof(a)/sizeof(a[0]))
```

But in C++, we should be able to do better. We have `constexpr` and templates, so we can use them:

```
template <typename T, size_t N>  
constexpr auto array_size(T (&)[N]) -> size_t {  
    return N;  
}
```

This seems like it should be a substantial improvement, yet it has surprising limitations:

```
void check(int const (&param)[3]) {  
    int local[] = {1, 2, 3};  
    constexpr auto s0 = array_size(local); // ok  
    constexpr auto s1 = array_size(param); // error  
}
```

The goal of this paper is to make that second case, and others like it, valid.

§ 1.1 Wait, why?

The reason is that in order for `array_size(param)` to work, we have to pass that reference to `param` into `array_size` - and that involves “reading” the reference. The specific rule we’re violating is 7.7 [expr.const]/5.12:

5 An expression *E* is a *core constant expression* unless the evaluation of *E*, following the rules of the abstract machine ([intro.execution]), would evaluate one of the following:

(5.12) — an *id-expression* that refers to a variable or data member of reference type unless the reference has a preceding initialization and either

(5.12.1) — it is usable in constant expressions or

(5.12.2) — its lifetime began within the evaluation of *E*;

The reason we violate the reference rule is due to the underlying principle that the constant evaluator has to reject all undefined behavior, so the compiler has to check that all references are valid.

This would be more obvious if our situation used pointers instead of references:

```
template <typename T, size_t N>
constexpr size_t array_size(T (*)(N)) {
    return N;
}

void check(int const (*param)[3]) {
    constexpr auto s2 = array_size(param); // error
}
```

This case *has* to be ill-formed, copying a function parameter during constant evaluation means it has to itself be a constant expression, and function parameters are not constant expressions - even in `constexpr` or `constexpr` functions.

But if the `param` case is ill-formed, why does the `local` case work? An unsatisfying answer is that... there just isn’t any rule in [expr.const] that we’re violating. There’s no lvalue-to-rvalue conversion (we’re not reading through the reference in any way yet) and we’re not referring to a reference (that’s the previous rule we ran afoul of). With the `param` case, the compiler cannot know whether the reference is valid, so it must reject. With the `local` case, the compiler can see for sure that the reference to `local` would be a valid reference, so it’s happy.

Notably, the rule we’re violating is only about *references*. We can’t write a function that takes an array by value, so let’s use the next-best thing: `std::array` and use the standard library’s `std::size` (cppref):

```
void check_arr_val(std::array<int, 3> const param) {
    std::array<int, 3> local = {1, 2, 3};
    constexpr auto s3 = std::size(local); // ok
    constexpr auto s4 = std::size(param); // ok
}
```

If `param` were a reference, the initialization of `s4` would be ill-formed (for the same reason as previously), but because it’s a value, this is totally fine.

So as long as you pass all your containers around by value, you're able to use `get` and use the size as a constant expression. Which is the kind of thing that's intellectually interesting, but also wildly impractical because obviously nobody's about to start passing all their containers around *by value*.

§ 1.2 Other Examples

Here are few other cases, which currently are ill-formed because of this reference-to-unknown rule.

From Andrzej Krzemienski:

Another situation where being able to use a reference to a non-core-constant object is when I am only interested in the type of the reference rather than the value of the object:

```
template <typename T, typename U>
constexpr bool is_type(U &&)
{
    return std::is_same_v<T, std::decay_t<U>>;
}
```

So that I can use it like this:

```
auto visitor = [](auto&& v) {
    if constexpr(is_type<Alternative1>(v)) {
        // ...
    } else if constexpr(is_type<Alternative2>(v)) {
        // ...
    }
};
```

I can do it with a macro:

```
#define IS_TYPE(TYPE, EXPR) (std::is_same_v<TYPE,
    std::decay_t<decltype(EXPR)>>)
```

From Jonathan Wakely:

```
auto rando(std::uniform_random_bit_generator auto& g)
{
    if constexpr (std::has_single_bit(g.max() - g.min()))
        // ...
    else
        // ...
}
```

The concept requires that `g.max()` and `g.min()` are constexpr static member functions, so this should work. And if I did it with an object of that type, it would work. But because `g` is a reference, it's not usable in a constant expression. That makes it awkward to refactor code into a function (or function template), because what worked on the object itself doesn't work in a function that binds a reference to that object.

I can rewrite it as something like:

```
using G = remove_reference_t<decltype(g)>;
if constexpr (std::has_single_bit(G::max() - G::min()))
```

Or avoid abbreviated function syntax so I have a name for the type:

```
template<std::uniform_random_bit_generator G>
auto rando(G& g)
{
    if constexpr (std::has_single_bit(G::max() - G::min()))
    }
```

But it's awkward that the first version doesn't Just Work.

Another from me:

I have a project that has a structure like:

```
template <typename... Types>
struct Widget {
    struct Config : Types::config... {
        template <typename T>
        static constexpr auto sends(T) -> bool {
            return std::is_base_of_v<typename T::config, Config>;
        }
    };

    Config config;
};
```

With the intent that this function makes for a nice and readable way of doing dispatch:

```
void do_configuration(auto& config) {
    // the actual type of config is... complicated

    if constexpr (config.sends(Goomba{})) {
        // do something
    }
    if constexpr (config.sends(Paratroopa{})) {
        // do something else
    }
}
```

Except this doesn't work, and I have to write:

```
void do_configuration(auto& config) {
    using Config = std::remove_cvref_t<decltype(config)>;

    if constexpr (Config::sends(Goomba{})) {
        // ...
    }
}
```

Which is not really "better."

What all of these examples have in common is that they are using a reference to an object of type `T` but do not care at all about the identity of that object. We're either querying properties of the type, invoking static member functions, or even when invoking a non-static member function (as in `std::array::size`), not actually accessing any non-static data members. The result would be the same for every object of type `T`... so if the identity doesn't change the result, why does the lack of identity cause the result to be non-constant? It's very much constant.

§ 2 Proposal

The proposal is to allow these cases to just work. That is, if during constant evaluation, we run into a reference with unknown origin, this is still okay, we keep going. If we ever perform an operation that actually *needs* this address, fail at that point.

Some operations are allowed to propagate a reference-to-unknown node (such as class member access or derived-to-non-virtual-base conversions). But most operations are definitely non-constant (such as lvalue-to-rvalue conversion, assignment, any polymorphic operations, conversion to a virtual base class, etc.). This paper is *just* proposing allowing those cases that work irrespective of the value of the reference (i.e. those that are truly constant), so any operation that depends on the value in any way needs to continue to be forbidden.

Notably, this paper is definitively *not* proposing any kind of short-circuiting evaluation. For example:

```
constexpr auto g() -> std::array<int, 10>&;
static_assert(g().size() == 10);
```

This check still must evaluate `g()`, which may or may not be a constant expression in its own right, even if `g().size()` is “obviously” 10. This paper is focused solely on those cases where we have an *id-expression* of reference type.

§ 2.1 Implementation Experience

I've implemented this in EDG at least to the extent that the test cases pretend in this paper all pass, whereas previously they had all failed.

§ 2.2 Other not-quite-reference examples

There are a few other closely related examples to consider for how to word this proposal. All of these are courtesy of Richard Smith.

We generally assume the following works:

```
auto f() {
    const int n = 5;
    return [] { int arr[n]; };
}
```

but `n` might not be in its lifetime when it's read in the evaluation of `arr`'s array bound. So we need to add wording to actually make that work.

Then there are further lifetime questions. The following example is similar to the other examples presented earlier:

```
struct A { constexpr int f() { return 0; } };
struct B : A {};
void f(B &b) { constexpr int k = b.f(); }
```

But this one is a bit different:

```
struct A2 { constexpr int f() { return 0; } };
struct B2 : virtual A2 {};
void f2(B2 &b) { constexpr int k = b.f(); }
```

Here, we convert `&b` to `A2*` and that might be undefined behavior (as per [class.ctor]/3). But this case seems similar enough to the earlier cases and should be allowed: `b.f()` is a constant, even with a virtual base. We need to ensure then that we consider references as within their lifetimes.

§ 2.3 Lifetime Dilemma

If we go back to this example:

```
extern B2 &b;
constexpr int k = b.f();
```

It seems reasonable to allow it, having no idea what the definition of `b` is. But what if we *do* see the definition of `b`, and it's:

```
union U { char c; B2 b2; };
constexpr U u = {.c = 0};
B2 &b = const_cast<B2&>(u.b2);
```

Now we *know* `b` isn't within its lifetime. We added more information, and turned our constant expression into a non-constant expression?

However, there's a reasonable principle here: anything that has only one possible interpretation *with defined behavior* has that defined behavior for constant evaluation purposes. This is true of all the examples presented up until now.

§ 2.4 Still further cases

A different case is the following:

```
struct A { virtual constexpr int f() { return 0; } } a;
constexpr int k = a.f();
constexpr auto &ti = typeid(a);
constexpr void *p = dynamic_cast<void*>(&a);
```

Here, `A::f` is `virtual`. Which might make it seem constant, but any number of shenanigans could ensue — like placement-new-ing a derived type (of the same size) over `a`. So all of these should probably remain non-constant expressions.

Perhaps the most fun example is this one:

```
extern const int arr[];
constexpr const int *p = arr + N;
constexpr int arr[2] = {0, 1};
constexpr int k = *p;
```

Which every compiler currently provides different results (in order of most reasonable to least reasonable):

1. Clang says `arr+N` is non-constant if `N != 0`, and accepts with `N == 0`.
2. GCC says `arr+N` is always constant (even though it sometimes has UB), but rejects reading `*p` if `arr+N` is out of bounds.
3. ICC says `arr+N` is always constant (even though it sometimes has UB), but always rejects reading `*p` even if `arr+N` is in-bounds.
4. MSVC says you can't declare `arr` as non-constexpr and define it constexpr, even though there is no such rule

This, to me, seems like there should be an added rule in `[expr.const]` that rejects addition and subtraction to an array of unknown bound unless that value is 0. This case seems unrelated enough to the rest of the paper that I think it should just be a Core issue.

§ 2.5 Wording

We need to strike the 7.7 [\[expr.const\]](#)/5.12 rule that disallows using references-to-unknown during constant evaluation:

- 5 An expression `E` is a *core constant expression* unless the evaluation of `E`, following the rules of the abstract machine (`[intro.execution]`), would evaluate one of the following:

- (5.1) — [...]
- (5.7) — [...]
- (5.8) — an lvalue-to-rvalue conversion unless it is applied to
 - (5.8.1) — a non-volatile glvalue that refers to an object that is usable in constant expressions, or
 - (5.8.2) — a non-volatile glvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of `E`
- (5.9) — [...]
- (5.10) — [...]

- (5.11) — an invocation of an implicitly-defined copy/move constructor or copy/move assignment operator for a union whose active member (if any) is mutable, unless the lifetime of the union object began within the evaluation of E;
- (5.12) — ~~an *id-expression* that refers to a variable or data member of reference type unless the reference has a preceding initialization and either~~
- (5.12.1) — ~~it is usable in constant expressions or~~
- (5.12.2) — ~~its lifetime began within the evaluation of E;~~
- (5.13) — in a *lambda-expression*, a reference to `this` or to a variable with automatic storage duration defined outside that *lambda-expression*, where the reference would be an odr-use;
- (5.14) — [...]

And add a new rule to properly handle the lifetime examples shown in the previous section:

- * During the evaluation of an expression E as a core constant expression, all *id-expressions* that refer to an object or reference with automatic storage duration that is usable in constant expressions are treated as referring to a specific instance of that object or reference whose lifetime includes the entire constant evaluation.

§ 3 Acknowledgments

Thanks to Daveed Vandevoorde for the encouragement and help. Thanks to Richard Smith for carefully describing the correct rule on the reflector and helping provide further examples and wording. Thanks to Michael Park for pointing out the issue to me, Tim Song for explaining it, and Jonathan Wakely for suggesting I pursue it.